

Projet Optimisation Combinatoire

Master 1 MIAGE (Mixte)

Optimisation des tournées en entrepôt : Hybridation du problème du Voyageur de Commerce (TSP) et du Clustering

Rapport réalisé du Mars 2026 au Avril 2026

présenté par

Kenza MENAD, Lyna BAOUCHE, Dyhia SELLAH

Table des matières

1	Introduction	3
2	Modélisation et Stratégies de Référence	4
2.1	Modélisation de l'environnement (Warehouse Environment)	4
2.1.1	Distance de Manhattan	4
2.2	Algorithme de Référence : Le Plus Proche Voisin (Baseline)	5
2.2.1	Principe et Évaluation	5
2.3	Optimisation par Algorithme Génétique (AG)	6
2.3.1	Paramétrage et Convergence	6
2.4	Analyse de Mise à l'Échelle (Scaling Analysis)	6
2.4.1	L'effondrement combinatoire	7
3	Intégration du Machine Learning	8
3.1	Les algorithmes de clustering évalués	8
3.2	Sélection automatique du nombre de clusters k	8
3.3	Métriques d'évaluation des clusterings	9
3.4	Résultats et comparaison	9
3.5	Visualisations	10
3.6	Choix de ML - K-Means	11
3.7	Intégration avec la métaheuristique	12
4	Métaheuristiques et Optimisation	13
4.1	Implémentation des Algorithmes	13
4.1.1	Algorithme des Colonies de Fourmis (ACO)	13
4.1.2	Algorithme Génétique (AG)	13
4.2	Résultats et Comparaison des Approches	14
4.2.1	Analyse des performances	14
5	Conclusion	15
6	Ressources et code source	16

Chapitre 1

Introduction

La préparation de commandes, ou *Order Picking*, est l'une des opérations les plus coûteuses dans la gestion d'un entrepôt. Elle représente en moyenne 55% des coûts opérationnels d'un entrepôt, et consiste à collecter des articles dispersés dans différentes zones pour constituer des commandes clients. Optimiser les déplacements du préparateur permet de réduire significativement ces coûts et d'améliorer la productivité.

Ce problème peut être modélisé comme un **Problème du Voyageur de Commerce** (TSP — *Travelling Salesman Problem*) : le préparateur doit visiter un ensemble de points (les articles à collecter) en parcourant la distance la plus courte possible, en partant et en revenant au dépôt. Cependant, lorsque le nombre d'articles est élevé (environ 119 articles dans notre cas), l'espace de recherche explose il y a $119!$ tournées possibles ce qui rend une résolution exacte impossible en temps raisonnable.

Pour faire face à cette complexité, nous proposons une **approche hybride** combinant deux familles de méthodes complémentaires :

- Le **Machine Learning** (clustering K-Means) pour décomposer intelligemment le problème en sous-problèmes plus petits, en regroupant les articles géographiquement proches en batches cohérents.
- Les **Métaheuristiques** (Algorithme des Colonies de Fourmis et Algorithme Génétique) pour optimiser les tournées à l'intérieur de chaque batch, et comparer leurs performances.

L'idée centrale est d'utiliser le clustering **en amont** de la métaheuristique : plutôt que de résoudre un TSP global de 119 points, on résout 4 sous-TSP d'environ 30 articles chacun, ce qui réduit drastiquement la complexité du problème.

Chapitre 2

Modélisation et Stratégies de Référence

2.1 Modélisation de l’environnement (Warehouse Environment)

La première étape de notre travail a consisté à créer une simulation fidèle des contraintes physiques d’un entrepôt. Nous avons modélisé cet espace sous la forme d’une grille orthogonale de 200×200 unités.

2.1.1 Distance de Manhattan

Contrairement à une distance à vol d’oiseau (Euclidienne), les préparateurs en entrepôt se déplacent le long d’allées perpendiculaires. Nous avons donc implémenté la **distance de Manhattan** (L_1 norm) pour calculer tous les déplacements :

$$d(A, B) = |x_A - x_B| + |y_A - y_B| \quad (2.1)$$

Cette modélisation garantit que nos résultats d’optimisation sont transposables à une configuration réelle de rayonnages. Le dépôt, point de départ et d’arrivée de chaque tournée, a été fixé aux coordonnées $(0, 0)$.

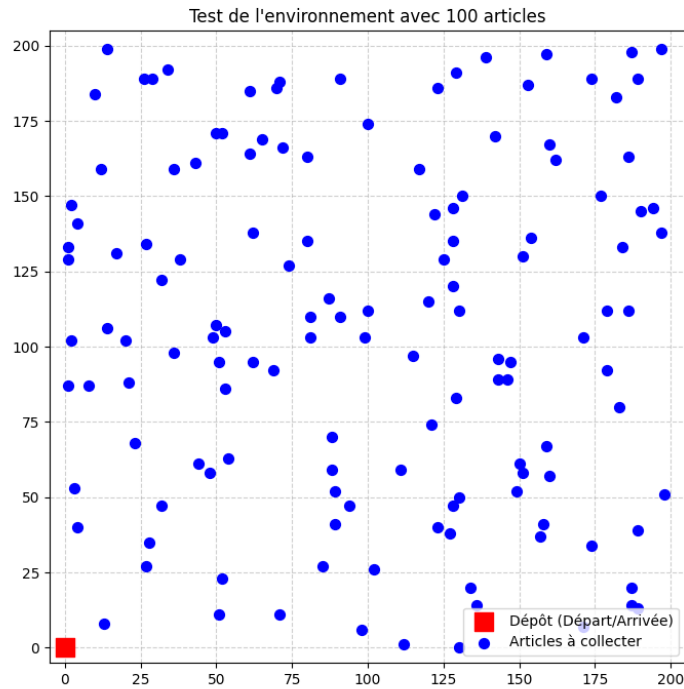


FIGURE 2.1 – Visualisation de l’environnement de l’entrepôt avec le dépôt et une vague de commandes générée aléatoirement.

2.2 Algorithme de Référence : Le Plus Proche Voisin (Baseline)

Afin d’évaluer l’efficacité de nos métaheuristiques, nous avons implémenté un algorithme "Glouton" (Nearest Neighbor).

2.2.1 Principe et Évaluation

À chaque étape, le préparateur se rend à l’article le plus proche de sa position actuelle. Cet algorithme nous sert de ligne de base (Baseline). Comme le montre le tableau 2.1, sa complexité algorithmique très faible lui permet de s’exécuter de manière quasi-instantanée, même sur de gros volumes.

Taille de la commande (Articles)	Distance Totale (Unités)	Temps d’exécution (s)
10	284	0,00012
30	642	0,00045
50	918	0,00103
100	1340	0,00350

TABLE 2.1 – Évaluation des performances de la Baseline selon la taille du problème.

Limites : Bien qu’extrêmement rapide, cet algorithme manque de vision globale. Il a tendance à "s’enfermer" aux extrémités de l’entrepôt, générant un long et coûteux

trajet de retour vers le dépôt.

2.3 Optimisation par Algorithme Génétique (AG)

En complément des colonies de fourmis, nous avons développé un **Algorithme Génétique** (AG) pour optimiser les tournées locales.

2.3.1 Paramétrage et Convergence

L'algorithme simule l'évolution naturelle (population de 100 individus, 300 générations, taux de mutation de 5 %). Les opérateurs génétiques utilisés sont la sélection par tournoi, le croisement de type *Order Crossover* (OX) adapté au TSP, et la mutation par échange (*Swap*).

La figure 2.2 illustre la courbe de convergence de notre Algorithme Génétique face à la Baseline sur une commande de taille moyenne.

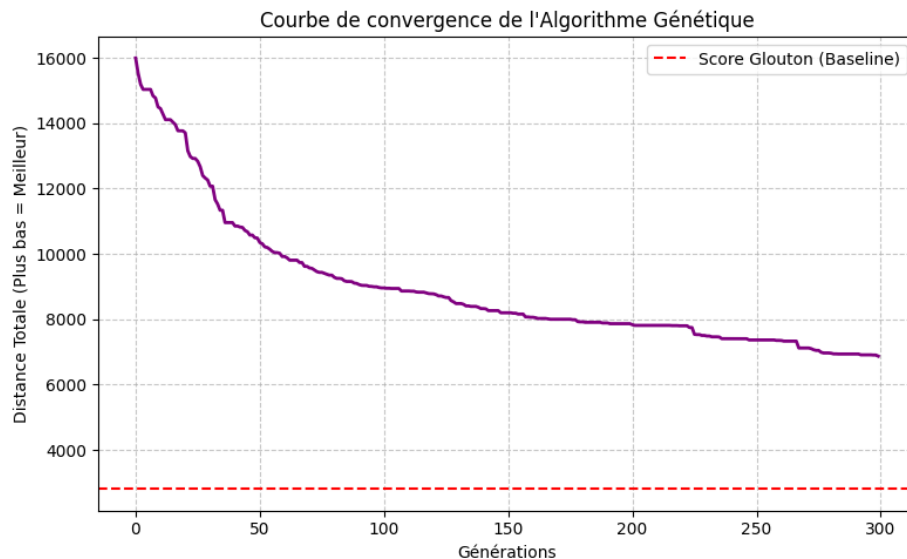


FIGURE 2.2 – Courbe de convergence de l'Algorithme Génétique par rapport au score Glouton.

2.4 Analyse de Mise à l'Échelle (Scaling Analysis)

Pour justifier l'intervention du Machine Learning (K-Means) dans notre pipeline, nous avons mené une expérience de résistance à l'échelle de notre Algorithme Génétique, détaillée dans le tableau 2.2.

Scénario	Distance Baseline	Distance AG	Gain/Perte (%)
Phase 1 : Petite commande (25 arts.)	512	435	+ 15,0 %
Phase 2 : Vague complète (130 arts.)	1680	2105	- 25,2 %

TABLE 2.2 – Comparaison des performances Baseline vs Algorithme Génétique selon le volume.

2.4.1 L'effondrement combinatoire

Les résultats révèlent un comportement critique : sur un petit volume (25 articles), l'AG surpasse la Baseline en trouvant des raccourcis globaux. Cependant, lorsque nous lui soumettons l'entrepôt complet (130 articles), l'AG s'effondre. L'espace de recherche devient si vaste ($130!$ permutations) que l'algorithme ne parvient plus à converger vers une solution acceptable dans un temps imparti, finissant par faire pire que le Glouton basique.

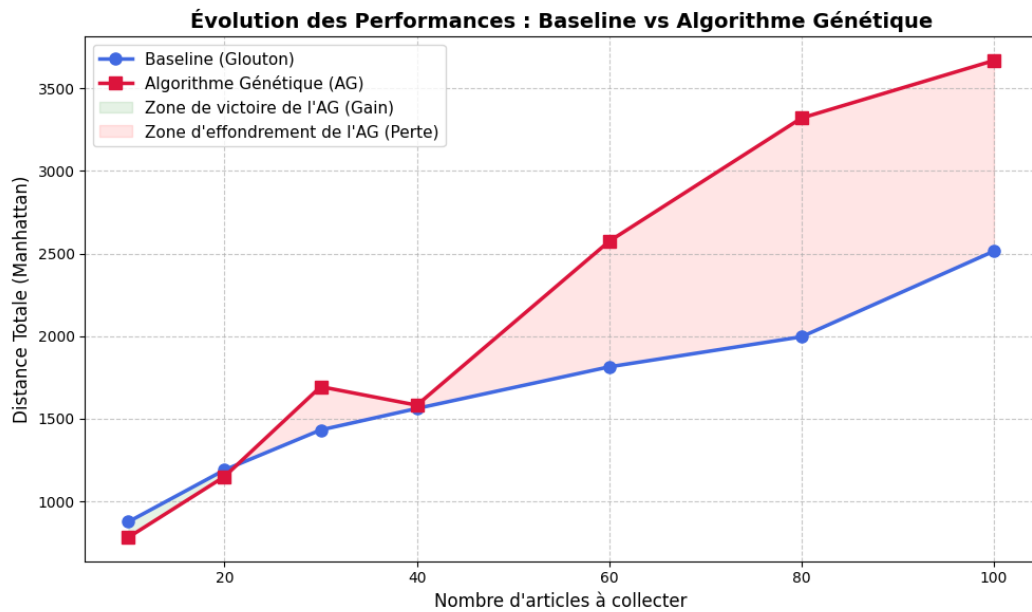


FIGURE 2.3 – Évolution croisée des performances mettant en évidence l'effondrement de l'AG face à la complexité combinatoire.

Conclusion intermédiaire : Cette analyse démontre formellement qu'une métaheuristique seule ne peut pas absorber la charge d'un entrepôt entier. Cela justifie pleinement notre architecture finale : utiliser le **Clustering (Machine Learning)** pour découper l'entrepôt en petits lots (< 30 articles), afin de toujours maintenir nos métaheuristicues dans leur "zone de victoire" (Phase 1).

Chapitre 3

Intégration du Machine Learning

Notre problème consiste à minimiser la distance parcourue par un préparateur de commandes dans un entrepôt de 200×200 unités, avec 119 articles à collecter. Si on soumet directement ces points à une métaheuristique (ACO ou algorithme génétique), on obtient un TSP de grande taille : l'espace de recherche explose ($119!$ permutations possibles) et la convergence est très lente.

Notre solution : utiliser le Machine Learning en amont pour regrouper les articles géographiquement proches en batches. Chaque batch devient alors un sous-TSP de petite taille, que la métaheuristique résout rapidement.

3.1 Les algorithmes de clustering évalués

Le clustering est une méthode d'apprentissage non supervisé : elle regroupe des données en groupes homogènes (clusters) sans étiquette préalable. Ici, chaque point est la coordonnée (x, y) d'un article. Nous avons comparé quatre algorithmes de familles différentes :

- **K-Means** : algorithme de partitionnement itératif basé sur la minimisation de la variance intra-cluster (WCSS). Il assigne chaque point au centroïde le plus proche, puis recalcule les centroïdes, et répète jusqu'à convergence.
- **Agglomerative Clustering** : algorithme hiérarchique ascendant. Il commence avec chaque point dans son propre cluster, puis fusionne progressivement les clusters les plus proches selon un critère de liaison (ici : Ward).
- **GMM — Gaussian Mixture Model (EM)** : modèle probabiliste qui suppose que les données sont générées par un mélange de distributions gaussiennes. L'algorithme Expectation-Maximization (EM) estime les paramètres de chaque gaussienne.
- **DBSCAN — Density-Based Spatial Clustering** : algorithme basé sur la densité. Il regroupe les points denses en clusters et marque comme bruit les points isolés. Il ne nécessite pas de spécifier k à l'avance.

3.2 Sélection automatique du nombre de clusters k

K-Means, Agglomerative et GMM nécessitent de fixer k avant l'exécution. Pour choisir k automatiquement, nous utilisons le **Silhouette Score**, calculé pour k allant de 2 à 8, et nous retenons la valeur qui le maximise.

Le Silhouette Score mesure à quel point chaque point est bien placé dans son cluster :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (3.1)$$

où $a(i)$ est la distance moyenne au sein du cluster (cohésion) et $b(i)$ est la distance au cluster voisin le plus proche (séparation). Un score proche de 1 indique des clusters bien séparés. Sur nos données (119 articles, `seed=42`), le score est maximal pour $k = 4$ avec un Silhouette Score de 0,404.

3.3 Métriques d'évaluation des clusterings

Pour comparer les algorithmes de façon rigoureuse, nous utilisons trois métriques complémentaires :

Métrique	Sens	Ce que ça mesure dans notre contexte
Silhouette Score	↑ mieux	Séparation entre clusters. Score élevé = articles d'un même batch proches → tournées locales courtes.
Davies-Bouldin	↓ mieux	Compacité des clusters par rapport à leur distance mutuelle. Index faible = clusters denses et bien séparés.
Calinski-Harabasz	↑ mieux	Densité intra-cluster vs dispersion inter-clusters. Score élevé = clusters bien distincts.
Temps d'exécution (ms)	↓ mieux	Durée du pré-traitement ML. Doit rester négligeable devant la métaheuristique.

TABLE 3.1 – Métriques d'évaluation des algorithmes de clustering

3.4 Résultats et comparaison

Les expériences ont été menées sur un entrepôt de 200×200 unités, 119 articles, `seed=42`, $k=4$, sous Python 3.13 / scikit-learn. Voici les résultats obtenus :

Algorithme	k	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	Bruit	Temps (ms)
K-Means	4	0,404	0,800	115,33	0 %	37,37
Agglomerative	4	0,361	0,810	96,40	0 %	39,25
GMM (EM)	4	0,390	0,831	109,21	0 %	30,05
DBSCAN	1	N/A	N/A	N/A	0,8 %	8,83

TABLE 3.2 – Comparaison des quatre algorithmes de clustering (119 articles, $k=4$, `seed=42`).

- **K-Means** obtient le meilleur Silhouette Score (0,404) et le meilleur Calinski-Harabasz (115,33), confirmant que ses clusters sont à la fois compacts et bien séparés géographiquement. Son temps d'exécution de 37,4 ms est raisonnable et son partitionnement est déterministe (grâce à la graine aléatoire fixée). C'est l'algorithme retenu pour notre pipeline.
- **Agglomerative Clustering** présente le meilleur Davies-Bouldin (0,810), ce qui signifie que ses clusters sont plus compacts en termes de dispersion interne. Cependant, son temps d'exécution est légèrement plus élevé que K-Means (39,25 ms).

ms) pour un gain de Silhouette négligeable ($-0,043$). Dans notre contexte où le pré-traitement doit être rapide, ce rapport coût/bénéfice est défavorable.

- **GMM (Gaussian Mixture Model)** donne les scores les plus faibles sur toutes les métriques de qualité (Silhouette 0,390, Davies-Bouldin 0,831, Calinski-Harabasz 109,21). Cela s'explique par le fait que le modèle gaussien n'est pas adapté à une distribution spatiale uniforme : les articles sont répartis aléatoirement dans l'entrepôt et ne suivent pas naturellement des distributions elliptiques gaussiennes.
- **DBSCAN a échoué dans cette configuration** : il n'a trouvé qu'un seul cluster ($k=1$), regroupant tous les articles en un seul batch, ce qui rend son utilisation impossible pour notre pipeline. Ce résultat illustre une limitation connue de DBSCAN : sa sensibilité au paramètre eps. Avec une distribution uniforme des articles, la densité est homogène sur tout l'entrepôt, rendant difficile la détection de zones denses distinctes.

3.5 Visualisations

La figure 1 montre côte à côte les résultats des quatre algorithmes sur les mêmes données. Pour K-Means et Agglomerative, on observe 4 zones géographiquement cohérentes. Pour GMM, les frontières entre clusters sont moins nettes, ce qui se reflète dans le Silhouette Score plus bas. DBSCAN regroupe tout en un seul cluster (orange), ce qui confirme l'échec de l'algorithme.

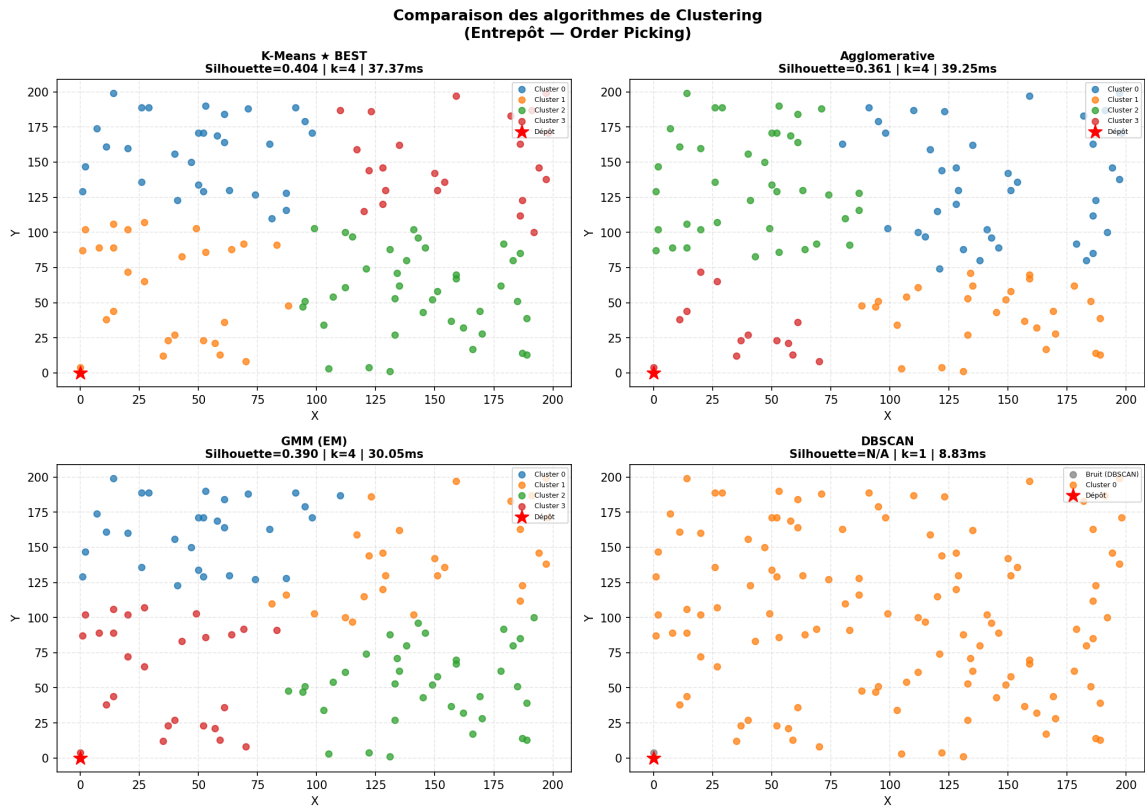


FIGURE 3.1 – Comparaison des quatre clusterings sur 119 articles (entrepôt 200×200).

La figure 2 montre que K-Means domine sur les métriques de qualité (Silhouette et Calinski-Harabasz), tout en maintenant un temps d'exécution modéré. DBSCAN

apparaît comme le plus rapide (8,83 ms), mais ses métriques de qualité sont toutes à N/A, ce qui le disqualifie.

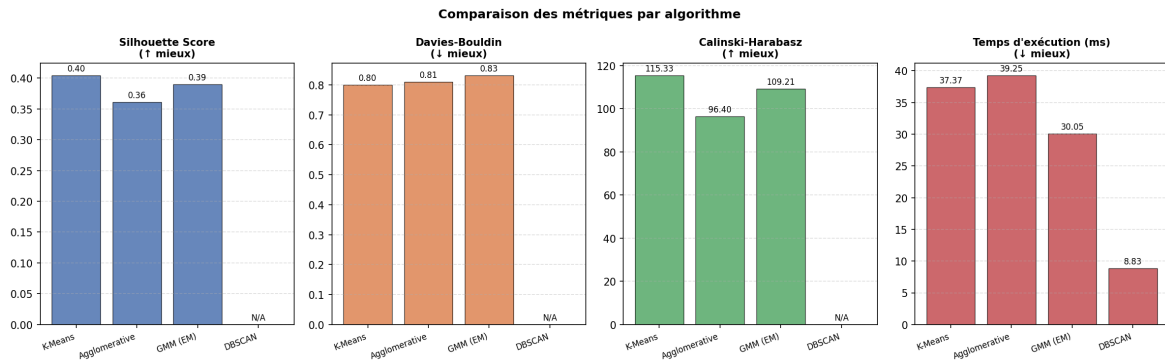


FIGURE 3.2 – Métriques comparatives par algorithme (Silhouette ↑, Davies-Bouldin ↓, Calinski-Harabasz ↑, Temps ↓).

3.6 Choix de ML - K-Means

Au terme de la comparaison, K-Means est retenu pour cinq raisons :

- Meilleure qualité de clustering (Silhouette 0,404 et Calinski-Harabasz 115,33) : les batches sont géographiquement cohérents.
- Résultats reproductibles grâce à la graine aléatoire fixée (seed=42), comme exigé dans le cahier des charges.
- **Intégration directe** : K-Means fournit nativement des labels et des centroïdes, exactement ce dont la métaheuristique a besoin.
- Pré-traitement rapide (37,4 ms), négligeable devant le temps de l'ACO/AG.
- **Cohérence avec la distance de Manhattan** : K-Means minimise la distance euclidienne, très proche de la distance de Manhattan dans un entrepôt à grille orthogonale.

La figure 3 présente en détail le clustering K-Means retenu. À gauche, la carte de l'entrepôt montre les 4 clusters avec leurs centroïdes et le nombre d'articles par cluster entre parenthèses. À droite, le diagramme en barres montre la distribution des tailles de clusters, avec la moyenne (environ 30 articles par batch) matérialisée par la ligne pointillée rouge. On observe que les clusters sont globalement équilibrés (de 23 à 39 articles), ce qui est favorable pour la métaheuristique : des batches de taille similaire garantissent des sous-TSP comparables en complexité, évitant qu'un seul batch ne concentre l'essentiel de la distance totale parcourue. Le cluster C0 (30 articles) est légèrement sur-représenté, ce qui pourrait être optimisé en ajustant k ou en ajoutant une contrainte de taille dans le clustering.

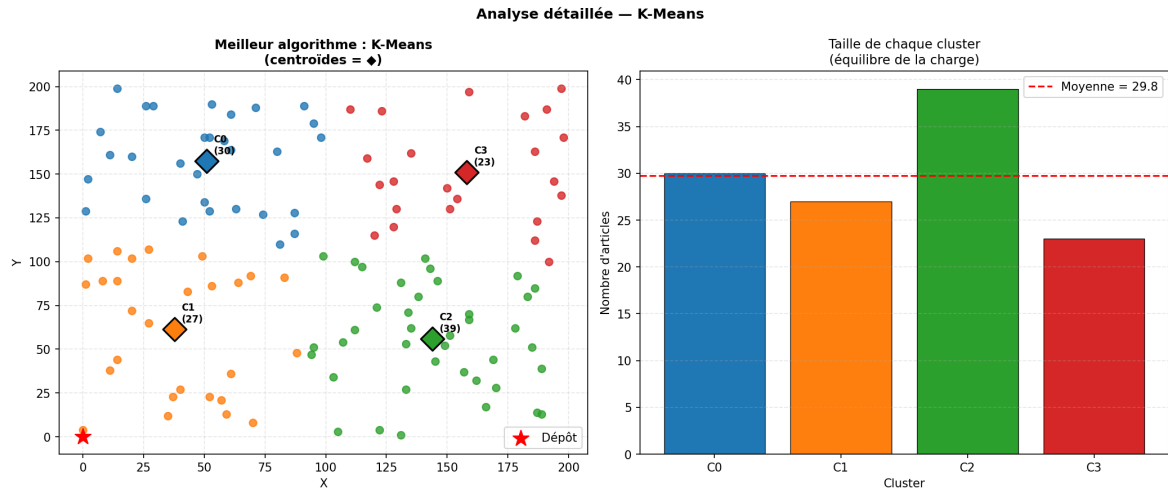


FIGURE 3.3 – K-Means ($k=4$) : carte de l'entrepôt avec centroïdes et distribution des tailles de clusters.

3.7 Intégration avec la métaheuristique

Le module ML transmet à la métaheuristique un dictionnaire Python structuré contenant : les batches (articles par cluster), les centroïdes, la matrice de distances de Manhattan entre centroïdes, et les coordonnées du dépôt. La métaheuristique n'a donc plus à traiter 119 points d'un seul coup, mais 1 TSP de 4 nœuds + 4 sous-TSP d'environ 30 articles chacun.

Gain de complexité : Sans clustering : $119!$ permutations. Avec clustering : $4! + 4 \times 30!$ réduction drastique de l'espace de recherche.

Chapitre 4

Métaheuristiques et Optimisation

Dans la continuité du pré-traitement par clustering K-Means, qui a permis de diviser notre problème initial de 69 articles en 2 sous-problèmes de type TSP de taille plus réduite, nous appliquons différentes métaheuristiques afin de déterminer la tournée optimale du préparateur au sein de chaque lot.

4.1 Implémentation des Algorithmes

4.1.1 Algorithme des Colonies de Fourmis (ACO)

L'algorithme des colonies de fourmis (ACO) est une métaheuristique bio-inspirée particulièrement adaptée au problème du Voyageur de Commerce (TSP). Dans notre implémentation, chaque "fourmi" artificielle construit une solution probabiliste en tenant compte de deux facteurs principaux pour se déplacer d'un point i vers un article j :

- **La visibilité** (η_{ij}) : définie comme l'inverse de la distance de Manhattan entre deux points, favorisant les déplacements courts.
- **La trace de phéromone** (τ_{ij}) : représentant l'historique des meilleurs chemins trouvés lors des itérations précédentes.

La probabilité de transition P_{ij} est définie par :

$$P_{ij} = \frac{(\tau_{ij})^\alpha \cdot (\eta_{ij})^\beta}{\sum_k (\tau_{ik})^\alpha \cdot (\eta_{ik})^\beta}$$

À chaque itération, les phéromones subissent une évaporation, tandis que les meilleures solutions renforcent les arcs empruntés. Ce mécanisme permet à l'algorithme de converger progressivement vers des solutions optimales.

4.1.2 Algorithme Génétique (AG)

Afin d'enrichir notre étude comparative, nous avons implémenté un Algorithme Génétique (AG). Cette approche évolutionniste manipule une population de solutions candidates (chromosomes), chacune représentant un chemin complet.

Le problème du TSP impose des contraintes spécifiques, notamment l'absence de duplication ou d'omission d'articles. Pour garantir cela, nous avons utilisé l'opérateur de croisement **Order Crossover (OX)**, permettant de préserver l'ordre de visite tout en assurant la validité des solutions.

Par ailleurs, un mécanisme de mutation par inversion a été intégré afin de maintenir la diversité génétique et éviter la convergence prématurée vers des minima locaux.

4.2 Résultats et Comparaison des Approches

Afin d'évaluer l'impact du clustering et des différentes métaheuristiques, nous avons comparé trois approches sur un même environnement :

1. **Approche A (Classique)** : ACO appliqué sur l'ensemble des articles (sans clustering).
2. **Approche B (Hybride ACO)** : Clustering K-Means suivi d'une optimisation par ACO sur chaque cluster.
3. **Approche C (Hybride AG)** : Clustering K-Means suivi d'une optimisation par Algorithme Génétique.

4.2.1 Résultats expérimentaux

Les expérimentations ont été réalisées sur un entrepôt simulé de dimension 200x200 contenant 69 articles. Le processus de clustering a automatiquement sélectionné l'algorithme K-Means avec $k = 2$ clusters, basé sur un score de silhouette de 0.4458.

Les performances obtenues sont les suivantes :

- **Approche A (ACO classique)** :
 - Distance totale : 1792 unités
 - Temps de calcul : 1.45 secondes
- **Approche B (ACO hybride)** :
 - Distance totale : 1984 unités
 - Temps de calcul : 0.81 secondes
- **Approche C (AG hybride)** :
 - Distance totale : 4724 unités
 - Temps de calcul : 0.06 secondes

4.2.2 Analyse des performances

Les résultats obtenus mettent en évidence un compromis fondamental entre qualité de solution et temps de calcul.

L'approche classique (ACO sans clustering) fournit la meilleure distance, car elle considère l'ensemble des articles dans une seule tournée globale. Cependant, cette méthode présente un coût computationnel plus élevé, lié à la taille importante de l'espace de recherche.

L'introduction du clustering (Approche B) permet de réduire significativement le temps de calcul. Cette amélioration s'explique par la décomposition du problème en sous-problèmes indépendants plus simples à résoudre. En contrepartie, la distance totale augmente légèrement en raison des retours au dépôt entre les clusters.

L'algorithme génétique (Approche C) se distingue par un temps d'exécution extrêmement faible, mais produit des solutions de moindre qualité. Cela montre une convergence rapide mais insuffisamment précise pour ce type de problème.

Ainsi, trois comportements distincts émergent :

- L'ACO classique privilégie la qualité de solution.
- L'ACO hybride offre un bon compromis entre performance et rapidité.

— L'AG privilégie la vitesse au détriment de l'optimalité.

4.2.3 Interprétation des résultats

L'augmentation de la distance dans les approches hybrides ne constitue pas une faiblesse, mais résulte d'une modélisation plus réaliste du problème logistique. En effet, contrairement à l'approche classique, les solutions hybrides imposent des retours au dépôt, reflétant les contraintes de capacité d'un chariot.

Les résultats confirment également la pertinence de l'ACO pour les problèmes de routage spatial, grâce à son mécanisme d'apprentissage progressif par phéromones.

À l'inverse, l'algorithme génétique, bien que rapide, nécessite un réglage plus fin de ses paramètres pour atteindre des performances comparables.

Enfin, l'intégration du Machine Learning en amont constitue un levier majeur pour réduire la complexité du problème, rendant son traitement possible à grande échelle.

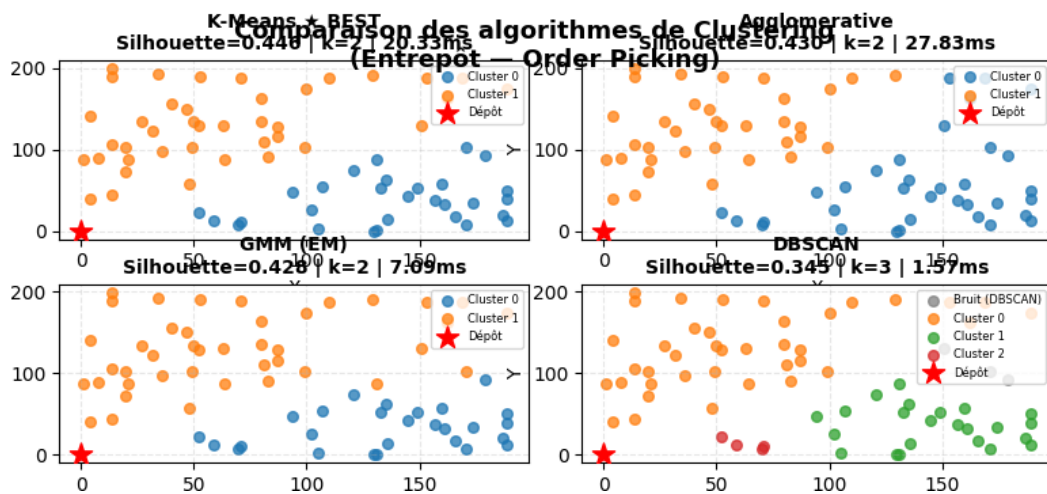


FIGURE 4.1 – Comparaison des algorithmes de clustering

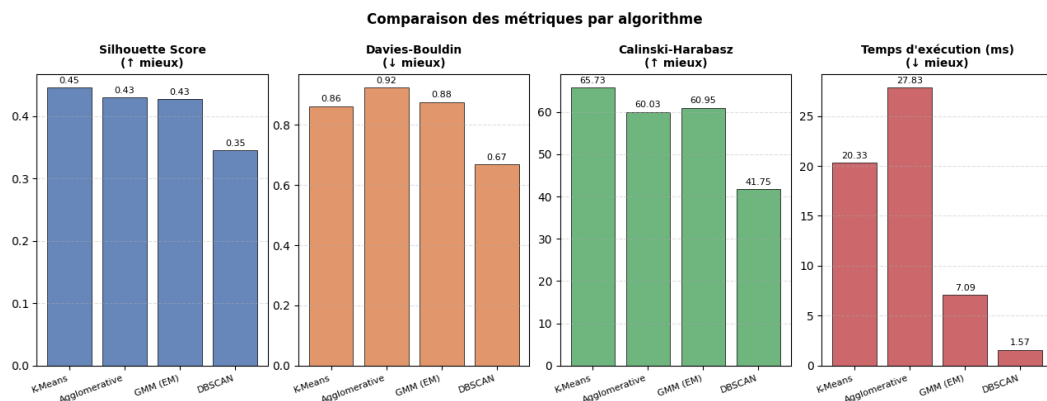


FIGURE 4.2 – Comparaison des métriques de performance

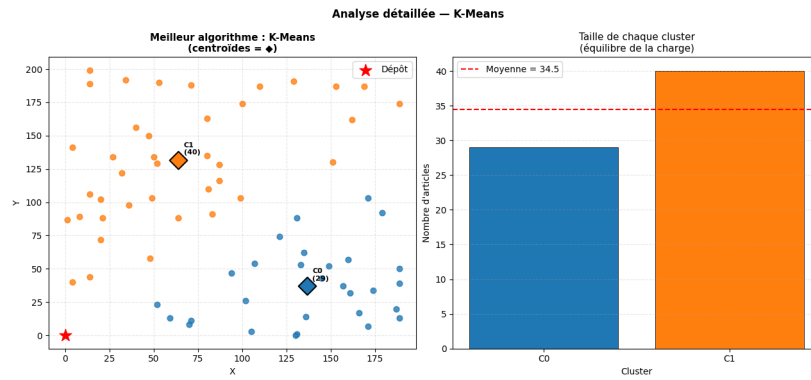


FIGURE 4.3 – Résultat du clustering optimal (K-Means)

Chapitre 5

Conclusion

Ce projet a permis de démontrer l'intérêt de l'hybridation entre le Machine Learning et l'Optimisation Combinatoire dans le cadre de la préparation de commandes.

L'utilisation du clustering K-Means a permis de segmenter efficacement l'espace de travail en zones géographiques cohérentes, réduisant ainsi la complexité globale du problème. Associée à des métaheuristiques performantes comme l'ACO, cette approche permet d'obtenir des solutions rapidement tout en conservant une qualité satisfaisante.

Cette architecture hybride constitue ainsi une solution robuste, scalable et adaptée aux contraintes réelles de la logistique moderne. Elle met en évidence qu'une approche intelligente basée sur la décomposition du problème offre un compromis optimal entre temps de calcul, qualité de solution et réalisme opérationnel.

Chapitre 6

Ressources et code source

L'intégralité du code source de ce projet est disponible sur GitHub :

`https://github.com/kenza-menad/Optimisation-combinatoire`